

Analisi di stabilità per la risoluzione numerica di un sistema di equazioni differenziali ordinarie stiff

Autore: Mirko Tagliabue
Progetto individuale

Indice:

Traccia del problema ed analisi teorica a priori	2
Calcolo degli errori	4
Il metodo di Radau-IIA.....	4
Il metodo di Gauss-Legendre su mesh non omogenea	6
Il metodo di Gauss-Legendre 2(6) adattivo	7
Il metodo BDF 2 ed il metodo BDF 3	9
Il metodo di Crank-Nicolson	11
Il metodo di Eulero Implicito.....	13
Il metodo di Pareschi Russo	14
Metodi RK espliciti ed espliciti adattivi	16

Traccia del problema ed analisi teorica a priori

Il problema in esame consiste nel risolvere numericamente un sistema di equazioni differenziali ordinarie lineari a coefficienti costanti della forma:

$$\underline{y}'(t) = M \cdot \underline{y}(t), \quad \text{con } \underline{y}(t_0) = [1, 1, 1, \dots, 10]^T =: \underline{y}_0,$$

da risolvere per $t \in [0, T]$, con T fissato e scelto nell'intervallo $[10, 10000]$.

La matrice M è definita come la matrice di dimensione 15×15 , avente come j -esimo elemento della diagonale principale il valore $-j^2$ e come elementi della sovradiagonale i valori 150:

$$M = \begin{bmatrix} -1 & 150 & 0 & \dots & 0 \\ 0 & -4 & 150 & \dots & 0 \\ 0 & 0 & -9 & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & 150 \\ 0 & 0 & \dots & 0 & -225 \end{bmatrix}$$

Osserviamo che M ha 15 autovalori distinti ed ha spettro: $\sigma(M) = \{-1, -4, \dots, -225\}$, in particolare, tutti e 15 i suoi autovalori sono reali negativi e pertanto il sistema di ODE (ordinary differential equations) in esame descrive una dinamica asintoticamente stabile per il teorema di Lyapunov, dove non si verificano fenomeni di divergenza come punti repulsori o punti di sella. L'origine è in effetti l'unico punto di equilibrio del sistema dinamico in esame.

Il problema viene risolto numericamente suddividendo l'intervallo $[t_0, T]$ in una partizione di sottointervalli aventi estremi $t_0 < t_1 < \dots < t_N = T$, dopodiché si approssima il valore della soluzione su ciascuno di tali estremi. Gli estremi degli intervalli vengono detti *nodi* ed il loro insieme viene detto *mesh*. La lunghezza di ogni sottointervallo è detta *passo di discretizzazione* h_n . Se tutti i nodi distano lo stesso passo h , la mesh è detta *uniforme* o anche *omogenea*.

Dall'analisi è noto che il sistema di ODE presenta una soluzione esatta della seguente forma:

$$\underline{y}(t) = \sum_{i=1}^{15} c_i \cdot e^{\lambda_i(t-t_0)} \cdot \underline{v}_i$$

dove λ_i rappresenta l' i -esimo autovalore della matrice M , $\underline{v}_i$ rappresenta un autovettore associato ad λ_i , ed c_i rappresenta un coefficiente numerico che dipende dal punto iniziale $\underline{y}_0$ e da $\underline{v}_i$.

Questa scrittura mette ben in luce come per $t \rightarrow +\infty$ la soluzione converge al vettore nullo (tutti i λ_i sono reali negativi), e che lo faccia con velocità di tipo esponenziale. Questo risulta infatti ben evidente già per tempi molto piccoli: anche solo per $t = 30$ il vettore soluzione ha norma euclidea dell'ordine di 10^{-5} , mentre per $t = 10,000$ tutte le componenti della soluzione esatta hanno valore indistinguibile da zero in aritmetica finita.

La scrittura della soluzione esatta mette anche in evidenza come tale soluzione possieda transitori lenti e transitori veloci. Nella combinazione lineare sopra descritta, per $t \rightarrow 0$ sarà il termine e^{-225t} ad essere quello predominante, mentre e^{-t} sarà il termine più trascurabile (ciò lo si può notare sviluppando la funzione $e^{\lambda t} = 1 + \lambda t + o(t)$ in serie di Taylor al primo ordine), d'altro canto, per $t \rightarrow +\infty$ accadrà l'esatto opposto e già per tempi t molto piccoli i valori della funzione e^{-225t} sono praticamente indistinguibili da zero. Ciò forza ad utilizzare un passo di discretizzazione h molto piccolo per descrivere accuratamente il problema nei tempi iniziali, al fine di catturare appieno il comportamento del transitorio più veloce, tuttavia, per descrivere il problema nei tempi grandi sarebbe sufficiente un passo h molto più ampio.

Ciò lo si può constatare anche osservando il coefficiente di stiffness:

$$C_{stiff} = \frac{\max_{i=1,\dots,15} |Re(\lambda_i)|}{\min_{i=1,\dots,15} |Re(\lambda_i)|} = 225 \gg 1$$

Un coefficiente di Stiffness pari ad oltre 200 forzerebbe un passo h molto piccolo per descrivere correttamente la soluzione nei primi istanti; tuttavia, se l'estremo superiore temporale T vale 10,000 e la mesh rimane uniforme, stiamo discretizzando l'intervallo $[t_0, T]$ in un numero elevatissimo di nodi intermedi solo per approssimare una funzione che a partire dall'istante $t = 30$ è pressoché indistinguibile dalla funzione identicamente nulla.

Tutte queste considerazioni insieme consentono di poter catalogare il presente problema come un problema di natura stiff.

I problemi stiff, forzando h ad essere estremamente piccolo, introducono una problematica di stabilità. In effetti, in linea puramente teorica, per $h \rightarrow 0^+$ non ci sono problemi: tutti i metodi numerici convergenti continuano ad essere convergenti, poiché comunque, la convergenza di un metodo non dipende dal problema in esame, ma solo dal metodo stesso; tuttavia, a livello implementativo avremo h finito e non " h appartenente ad ogni intorno di zero piccolo a piacere", ciò introduce un *errore di arrotondamento* che viene necessariamente sempre introdotto quando si lavora in aritmetica finita al calcolatore. Un metodo numerico instabile è un metodo numerico che propaga piccole perturbazioni fino a renderle enormi, un metodo numerico stabile è un metodo numerico che assorbe e smorza piccole o grandi perturbazioni numeriche, rendendole progressivamente sempre più piccole. Non tutti i tipi di stabilità sono uguali, si ricorda pertanto che:

$$L\text{-stabilità} \Rightarrow A\text{-stabilità} \Rightarrow \theta\text{-stabilità} \Rightarrow \text{zero-stabilità}$$

e che

$$\text{zero-stabilità} + \text{consistenza} \Rightarrow \text{convergenza}$$

Di contro, il prezzo da pagare è che i metodi più stabili sono anche quelli computazionalmente più onerosi, in effetti, si può dimostrare che tutti i metodi L-stabili ed A-stabili sono necessariamente di tipo implicito, e questo richiede che ad ogni passo si risolva un sistema di equazioni.

Prima di proseguire approfondiamo ancora il comportamento della soluzione esatta.

In generale, un sistema di equazioni differenziali ordinarie lineari a coefficienti costanti del tipo:

$$\underline{y}'(t) = M \cdot \underline{y}(t), \quad \underline{y}(t_0) = \underline{y}_0, \quad t \in (a, b)$$

ammette come soluzione esatta:

$$\underline{y}(t) = \exp(M \cdot (t - t_0)) \cdot \underline{y}_0, \quad t \in (a, b)$$

dove $\exp$ è in questo caso l'esponenziale di matrice, ovvero:

$$\exp(A) := \sum_{k=0}^{+\infty} \frac{A^k}{k!}$$

È possibile calcolare l'esponenziale di matrice mediante la funzione `expm` della libreria standard di MATLAB. Avere accesso alla soluzione esatta di fatto rende poco significativo il problema di trovare una soluzione numerica approssimata, tuttavia, dato che il nostro interesse è quello di studiare la stabilità numerica dei metodi, e non quello di trovare una soluzione allo specifico problema in esame, avere a disposizione la soluzione esatta ci consentirà di calcolare in modo concreto gli errori commessi, e non solo una loro stima per eccesso. Una volta appurata la stabilità di un metodo numerico, esso si potrà usare anche per approssimare equazioni differenziali di cui non è nota (o non è possibile trovare) una soluzione in forma chiusa.

Per rappresentare la soluzione esatta abbiamo pertanto utilizzato il comando `expm` di MATLAB. L'esponenziale di matrice risulta più preciso rispetto allo scrivere la soluzione esatta come:

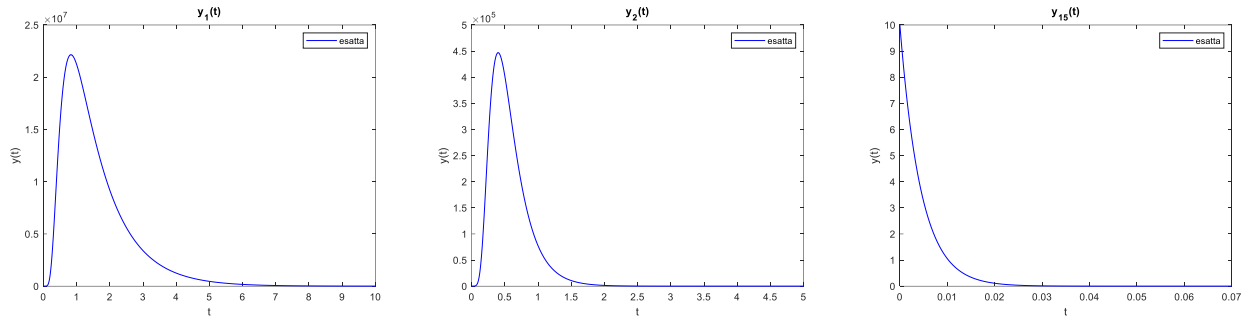
$$y(t) = \sum_{i=1}^{15} c_i \cdot e^{\lambda_i t} \cdot v_i, \text{ forma della soluzione che abbiamo introdotto a pagina precedente.}$$

La soluzione esatta sarà una funzione vettoriale della forma:

$$\underline{y}(t) = [y_1(t), y_2(t), \dots, y_{15}(t)]^T.$$

Nei codici e per tutta la presente relazione abbiamo scelto, per brevità, di rappresentare solo le componenti y_1, y_2 ed y_{15} , non avrebbe infatti avuto molto senso pratico rappresentarle tutte e 15.

La soluzione esatta è la seguente:



possiamo osservare che essa presenta un picco iniziale nei primissimi valori di t , dove raggiunge anche valori di 10^7 nella prima componente (e 10^5 nella seconda componente), ma dopo il picco iniziale essa decade molto velocemente. Si noti in particolare che la funzione y_{15} nella terza figura sopra è rappresentata solo fino a 0.07.

Calcolo degli errori

Nei passaggi seguenti abbiamo calcolato sia l'errore di discretizzazione globale, sia l'errore relativo. L'errore di discretizzazione globale è definito tramite la seguente formula:

$$e := \max_{n=0,\dots,N} e_n := \max_{n=0,\dots,N} \|y(t_n) - u(t_n)\|_2$$

dove $u(t)$ è la soluzione numerica ed $y(t)$ è la soluzione esatta. L'errore di discretizzazione globale gioca quindi un ruolo simile all'errore assoluto: è il massimo dello scarto in norma euclidea tra la soluzione esatta e quella numerica, ovverosia sarebbe: $e = \|y - u\|_\infty$

Abbiamo deciso di calcolare anche l'errore relativo perché le grandezze in gioco sono molto considerevoli, il picco della prima componente della soluzione esatta ha ordine di grandezza 10^7 , e pertanto anche l'errore relativo individua una informazione utile, esso si definisce come:

$$Err_{rel} := \frac{\max_{n=0,\dots,N} e_n}{\max_{n=0,\dots,N} \|y(t_n)\|_2}$$

cioè il rapporto tra il massimo errore di discretizzazione locale e la massima della norma della soluzione esatta.

Il metodo di Radau-IIA

Trattasi di un metodo di Runge Kutta basato su metodo di collocazione: l'idea di base di questi algoritmi consiste nel cercare un polinomio $P(t)$ di grado s che interpoli sui nodi della mesh la funzione associata al problema di Cauchy, utilizzando poi una tecnica dimostrativa basata sui polinomi di Lagrange è possibile verificare che utilizzando un'interpolazione con queste caratteristiche, i metodi di collocazione risultano essere dei metodi di Runge-Kutta. Scegliendo come polinomio interpolante proprio un polinomio di Radau di tipo IIA, si ottiene un metodo L-stabile a $s = 3$ stadi di ordine $p = 2s - 1 = 5$.

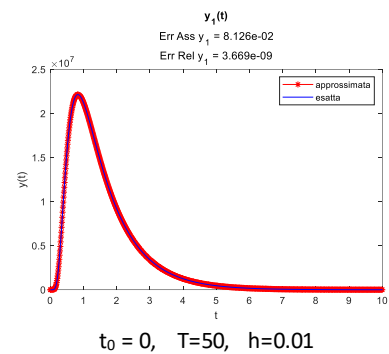
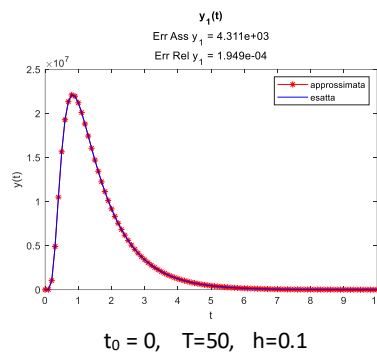
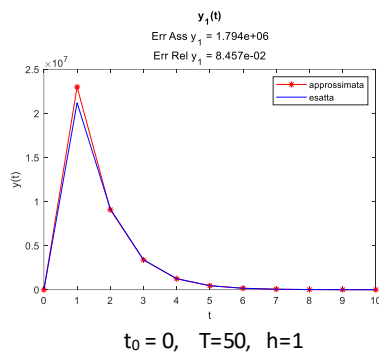
Il fatto che il metodo abbia ordine 5 significa che:

$$e = \max_{n=0,\dots,N} \|y(t_n) - u(t_n)\|_2 = O(h^5),$$

e che pertanto, diminuendo il passo h di un fattore 10, l'errore assoluto si abbatta di un fattore 10^5 .

Si tratta di un abbattimento notevole: non solo il metodo è incredibilmente stabile, ma è anche straordinariamente preciso. Lo svantaggio è che il metodo è anche costoso da un punto di vista computazionale: per ogni singolo nodo della mesh si richiede di risolvere un sistema 45×45 ; tuttavia, in questo particolare caso dove il sistema di ODE è lineare, non solo i sistemi da risolvere numericamente sono lineari, ma la matrice A che descrive il sistema $Ax_n = b_n$ è sempre la stessa per tutto il metodo, cambia solo il termine noto b_n , ciò consente di utilizzare la fattorizzazione LU una volta per tutte all'inizio ed abbattere così il costo computazionale richiesto per risolvere i sistemi ad un costo quadratico di operazioni macchina (*flops*).

Di seguito diamo alcuni grafici per rappresentare l'andamento dell'errore. I tre grafici seguenti mostrano la prima componente $y_1(t)$ del vettore soluzione $\underline{y}(t)$ e sono stati tutti e tre prodotti utilizzando come parametri $t_0 = 0$ e $T = 50$, inoltre nella prima immagine $h = 1$, nella seconda $h = 0.1$, nella terza $h = 0.01$. È decisamente soddisfacente osservare come con anche solo $h = 0.1$ e dunque $N = 501$ nodi (un numero quasi sempre irrisorio quando si parla di cardinalità di una mesh) il metodo produca un errore relativo di 10^{-4} , risultando quindi decisamente affidabile.



Diamo anche due tabelle degli errori, costruite scegliendo $T = 10$ e $T = 10,000$:

$T = 10$		
numero nodi	errore discretizz. globale	errore relativo
100	4.35928e+03	1.97120e-04
1,000	8.13346e-02	3.67199e-09
10,000	1.27448e-06	5.75384e-14
100,000	2.64496e-07	1.19411e-14

$T = 10000$		
numero nodi	errore discretizz. globale	errore relativo
100,000	4.35928e+03	1.97120e-04
1,000,000	8.13346e-02	3.67199e-09

I dati ottenuti sono coerenti con le aspettative teoriche: diminuendo h di un fattore 10, e dunque aumentando il numero di nodi di un fattore 10, l'errore commesso si abbatte di un fattore 10^5 . Ciò nonostante, l'errore assoluto non riesce a scendere al di sotto di 10^{-7} a causa del rumore di fondo numerico che si viene a produrre risolvendo centinaia di migliaia di sistemi di 45 equazioni in 45 incognite. Il metodo converge comunque abbastanza velocemente: lavorando con un milione di nodi il metodo impiega solo circa 2 minuti per restituire l'output (anche se comunque il tempo di esecuzione dipende fortemente dalle specifiche tecniche del computer utilizzato per eseguire il codice).

Si specifica inoltre che tutti gli algoritmi presentati in questo documento sono stati sviluppati *from scratch* (da zero) utilizzando unicamente la libreria standard di MATLAB. I codici risultano pertanto pienamente compatibili sia con le licenze base di MATLAB (senza la necessità di specifici toolbox a pagamento), sia con il software open-source GNU Octave.

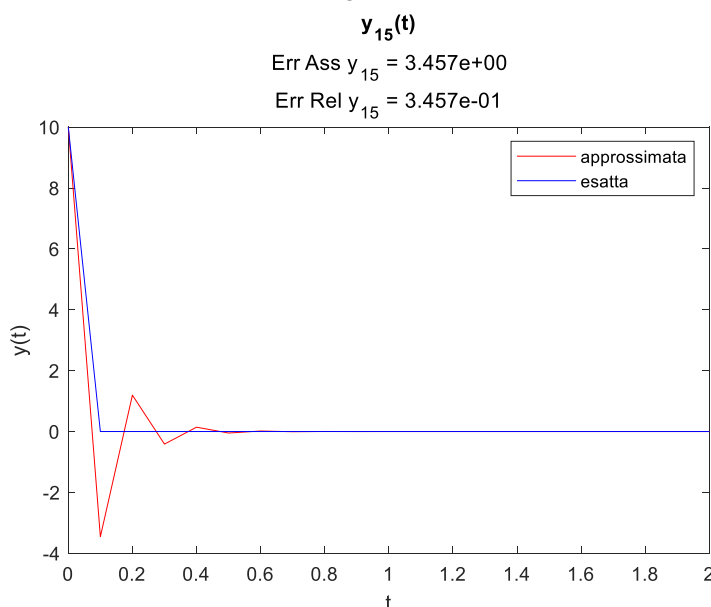
Il metodo di Gauss-Legendre su mesh non omogenea

Come abbiamo discusso in precedenza, i problemi stiff forzano ad utilizzare un passo eccessivamente piccolo necessario per descrivere accuratamente la soluzione solo su un numero molto limitato di nodi, mentre altrove nella mesh una tale densità di nodi è eccessiva ed assolutamente non necessaria. È qui che una mesh omogenea mostra i suoi limiti.

Implementando una mesh non omogenea si è deciso di discretizzare con un passo h_1 l'intervallo $[t_0, 20]$, con un passo h_2 l'intervallo $[20, 100]$, e con un passo h_3 l'intervallo $[100, T]$, con l'idea di prendere h_1 estremamente piccolo ed h_3 estremamente grande. Ovviamente, comunque, la cosa ha senso solo quando $t_0 < 20 - h_1$ e $T > 100 + h_3$.

L'idea di utilizzare una mesh non omogenea è vincente e consente di abbattere notevolmente il numero di nodi nella mesh, e di conseguenza l'errore macchina introdotto risolvendo i vari sistemi lineari. Di contro, il prezzo da pagare per avere una mesh non omogenea è che, essendo il passo h variabile, la matrice A che descrive i sistemi lineari non sarà più statica per tutto il metodo, ma dipenderà anche dal nodo in questione, questo non ci permetterà di utilizzare la fattorizzazione LU , aumentando di conseguenza il numero di *flops* necessari per risolvere i sistemi lineari. Tuttavia, il costo computazionale da pagare per rinunciare alla fattorizzazione LU è abbondantemente compensato dall'abbattere drasticamente il numero di sistemi lineari da risolvere.

Abbiamo pertanto implementato due metodi numerici su una mesh non omogenea, uno di questi è il metodo di Gauss-Legendre a $s = 3$ stadi, l'altro è il metodo di Crank-Nicolson. I metodi di Gauss-Legendre, così come i metodi di Radau-IIA, sono metodi di Runge Kutta basati sul metodo di collocazione, con l'accortezza che questa volta viene usata un'interpolazione polinomiale di tipo Gauss-Legendre. I metodi di Gauss-Legendre sono metodi A-stabili di ordine $p = 2s$, dove s è il numero di stadi utilizzato. Segue pertanto che, sebbene i metodi di Gauss-Legendre siano più precisi dei metodi di Radau ($p = 2s$ contro $p = 2s - 1$), essi sono meno stabili (A-stabili anziché L-stabili). Interessante è osservare che, sebbene in generale la A-stabilità sia più che sufficiente nella stragrande maggioranza delle applicazioni, un problema numerico con coefficiente di stiffness oltre i 200 è caratterizzato da una profonda instabilità, e che pertanto, prendendo un passo h_1 non abbastanza piccolo potrebbero comparire delle oscillazioni iniziali (che poi andrebbero a smorzarsi, come previsto teoricamente che succeda utilizzando dei metodi A-stabili). In effetti è ciò che accade: prendendo $h_1 = 0.1$ (che di per sé è irragionevolmente grande, spesso si opta per passi decisamente inferiori, specialmente nei punti di picco) si notano delle piccole oscillazioni negli istanti iniziali della soluzione approssimata $u_{15}(t)$.



$$t_0 = 0, \quad T = 10,000 \\ h_1 = 0.1, \quad h_2 = 1, \quad h_3 = 10$$

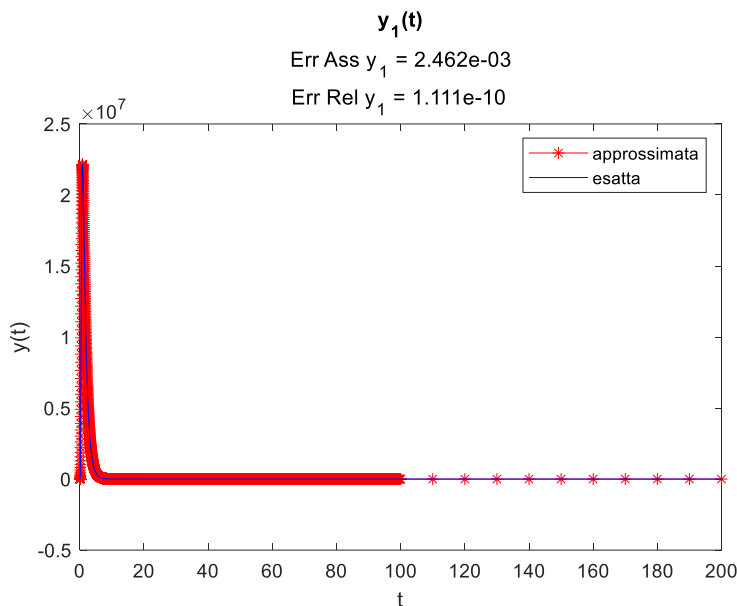
Un passo h_1 troppo grande introduce dei fenomeni di oscillazione nella componente u_{15} della soluzione approssimata a causa della mancanza di L-stabilità.

La A-stabilità del metodo provvede comunque a smorzare le oscillazioni.

Le oscillazioni compaiono solo sulla 15-esima componente poiché è la dinamica con autovalore più alto in modulo, per osservare oscillazioni anche sulle altre componenti bisognerebbe prendere un passo h_1 ancora più grande, che però introdurrebbe un errore assoluto eccessivo, rendendo la soluzione approssimata trovata priva di qualunque significato.

Non è un caso che oscillazioni sulle altre componenti siano difficili da osservare: la A-stabilità è una proprietà matematica decisamente forte e che comunque rende il metodo incondizionatamente assolutamente stabile.

La foto seguente mette in luce come appare la soluzione su una mesh non omogenea.



$$t_0 = 0, \quad T = 10,000$$

$$h_1 = 0.01, \quad h_2 = 0.1, \quad h_3 = 10$$

La mesh è più densa negli istanti iniziali e più sparsa in quelli finali.

Forniamo infine una tabella con gli errori fissati i parametri $t_0 = 0$ ed $T = 10,000$.

Num. nodi	Err. Ass	Err. Rel.	Scelta passi: h_1 in $[t_0, 20]$, h_2 in $[20, 100]$, h_3 in $[100, T]$
291	6.69894e+02	3.02915e-05	$h_1 = 0.1, \quad h_2 = 1, \quad h_3 = 1000$
2,900	1.66442e-02	7.51429e-10	$h_1 = 0.01, \quad h_2 = 0.1, \quad h_3 = 100$
20,180	5.34687e-07	2.41393e-14	$h_1 = 0.001, \quad h_2 = 1, \quad h_3 = 100$
21,791	1.08034e-07	4.87737e-15	$h_1 = 0.001, \quad h_2 = 0.1, \quad h_3 = 10$
100,000	6.69894e-02	3.02915e-05	$h = 0.1$, mesh omogenea
1,000,000	1.66442e-02	7.51429e-10	$h = 0.01$, mesh omogenea

È interessante osservare come, lavorare con 2,900 nodi in mesh non omogenea, oppure lavorare con 1,000,000 nodi in mesh omogenea, produca in realtà lo stesso errore.

È anche interessante confrontare questo metodo in mesh omogenea con quello di Radau-IIA: nei casi di $T = 10,000$, passi con $h = 0.1$ ed $h = 0.01$ si può infatti osservare che il metodo di Gauss-Legendre è più preciso di un fattore 10, coerente con la precisione teorica dei due metodi.

Il metodo di Gauss-Legendre 2(6) adattivo

I metodi adattivi sono metodi numerici in cui la mesh non è fissata a priori ma i suoi nodi sono calcolati in fase di esecuzione del programma, sulla base di una stima dell'errore. In ogni nodo si determina automaticamente quanto vale il passo di discretizzazione successivo. Sono molto spesso una scelta molto solida quando si ha a che fare con problemi stiff.

Il metodo di Gauss-Legendre 2(6) è un metodo di Runge Kutta implicito adattivo embedded, ciò significa che esistono due metodi di Gauss-Legendre, uno di ordine 2 ed uno di ordine 6, che condividono lo stesso Tableau di Butcher, eccezion fatta per il vettore b . Un metodo userà i pesi b e l'altro userà i pesi $\hat{b}$.

Il Tableau di Butcher dei due metodi è il seguente:

$\frac{1}{2} - \frac{\sqrt{15}}{10}$	$\frac{5}{36}$	$\frac{2}{9} - \frac{\sqrt{15}}{15}$	$\frac{5}{36} - \frac{\sqrt{15}}{30}$	<table><tr><td>$\underline{c}$</td><td>A</td></tr><tr><td></td><td>$\underline{b^T}$</td></tr><tr><td></td><td>$\underline{\widehat{b^T}}$</td></tr></table>	$\underline{c}$	A		$\underline{b^T}$		$\underline{\widehat{b^T}}$
$\underline{c}$	A									
	$\underline{b^T}$									
	$\underline{\widehat{b^T}}$									
$\frac{1}{2}$	$\frac{5}{36} + \frac{\sqrt{15}}{24}$	$\frac{2}{9}$	$\frac{5}{36} - \frac{\sqrt{15}}{24}$							
$\frac{1}{2} + \frac{\sqrt{15}}{10}$	$\frac{5}{36} + \frac{\sqrt{15}}{30}$	$\frac{2}{9} + \frac{\sqrt{15}}{15}$	$\frac{5}{36}$							
	$\frac{5}{18}$	$\frac{4}{9}$	$\frac{5}{18}$							
	$-\frac{5}{6}$	$\frac{8}{3}$	$-\frac{5}{6}$							

Il vantaggio è considerevole: risolvendo gli stessi sistemi lineari e cambiando solo i pesi b delle combinazioni lineari (il costo computazionale di ciò è assolutamente irrisorio) abbiamo a disposizione due soluzioni una A-stabile di ordine 6, ed un'altra di ordine 2.

Trattando la soluzione ottenuta col metodo di ordine superiore come se fosse la soluzione esatta e calcolando lo scarto tra le due soluzioni si ha a disposizione un valore che può essere considerato uno stimatore dell'errore assoluto, anche in quei casi dove la soluzione esatta non sia nota a priori. Se lo scarto è inferiore ad una tolleranza fissata si accetta il valore trovato inserendolo nella mesh e poi si procede a calcolare il nuovo passo h_n , in caso alternativo, si ripete il calcolo riducendo il passo. Infine, si tiene memorizzata come soluzione numerica la soluzione fornita dal metodo di ordine 6.

Segue una tabella contenente i vari errori al variare della tolleranza, ($t_0 = 0$, $T = 10,000$):

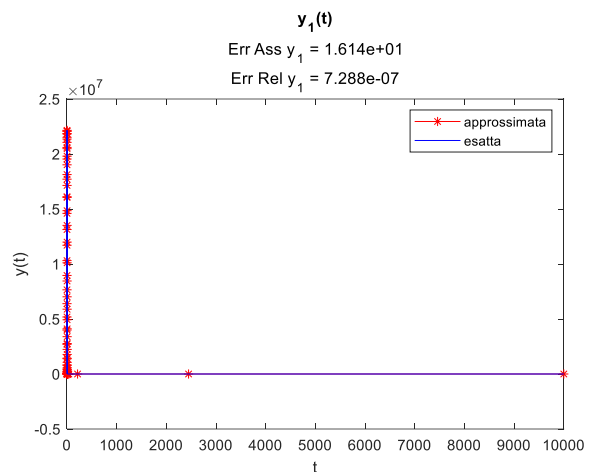
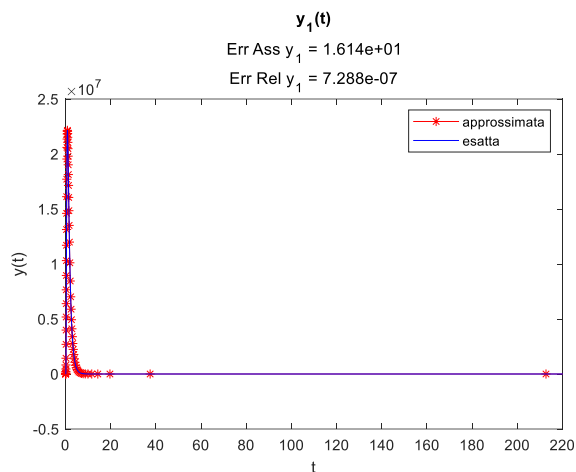
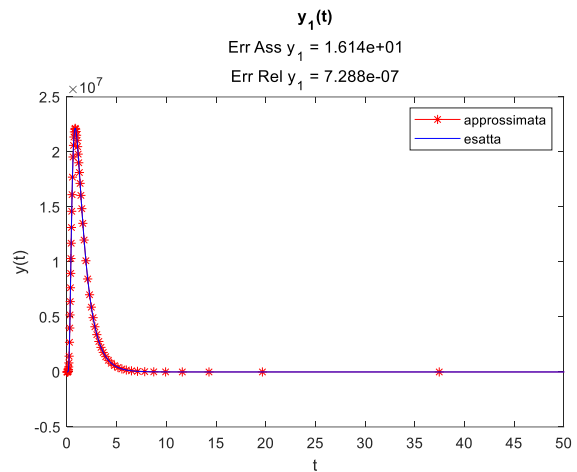
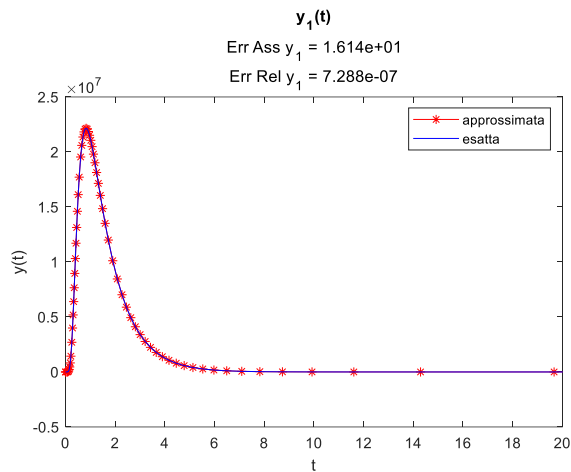
Tolleranza	Numero nodi	Err. Ass.	Err. Rel.	h min	h max
1e+5	38	1.32016e+02	5.97904e-06	0.001	7515.21744
1e+4	70	1.62036e+01	7.31537e-07	0.001	7552.67047
1e+3	138	3.22138e-01	1.45438e-08	0.001	7452.33204
1e+2	281	2.18029e-02	9.84334e-10	0.001	5030.59686
1e+1	589	7.42305e-04	3.35125e-11	0.001	8677.20076
1e+0	1,254	7.42405e-05	3.35171e-12	0.00096	8649.32396
1e-1	2,687	6.57684e-06	2.96922e-13	0.00045	7031.06471
1e-2	5,774	7.13678e-07	3.22201e-14	0.00021	4776.71363
1e-3	12,423	1.86265e-07	8.40921e-15	0.00010	7578.50364
1e-4	26,748	1.37839e-07	6.22294e-15	0.00004	7133.44752
1e-5	57,609	2.71946e-07	1.22775e-14	0.00002	6734.75288
1e-6	124,095	4.09783e-07	1.85003e-14	0.00001	8444.49846

Un fatto interessante da osservare è che con tolleranze di 10^{-5} e 10^{-6} l'errore assoluto ricomincia a risalire da $1.378 \cdot 10^{-7}$ ottenuto con $toll = 10^{-4}$, fino ad $4.098 \cdot 10^{-7}$ quando $toll = 10^{-6}$. Ciò è un fatto generale di analisi numerica: diminuendo eccessivamente il passo di discretizzazione h l'errore introdotto dalle operazioni macchina (*errore di arrotondamento*) inizia a diventare predominante sull'errore introdotto dal metodo (*errore di discretizzazione*). È un limite ben preciso: diminuendo sempre più h , l'errore non continua a diminuire all'infinito, al contrario, ricomincia a salire. In effetti esiste un punto di minimo dell'*errore totale* commesso (somma dei due errori sopra citati) ed un h_{opt} ottimo che minimizza tale errore.

È inoltre particolarmente interessante osservare come anche fissando una tolleranza enorme di 10^4 , il metodo numerico produca, con solo 70 nodi, un errore assoluto dell'ordine delle decine per approssimare variabili che viaggiano su grandezze nell'ordine delle decine di milioni.

Il costo computazionale è assolutamente irrisorio, richiedendo al più qualche secondo per tolleranze inferiori ad 10^{-4} , diventando pressoché istantaneo per tolleranze superiori a 10^{-2} .

Segue una progressione di immagini che descrive con diversi livelli di zoom l'approssimazione di $y_1(t)$, con $t_0 = 0$, $T = 10,000$, $toll = 10^4$, $Num\ nodi = 70$, le quattro immagini seguenti sono viste rispettivamente con zoom negli intervalli: $[0, 20]$, $[0, 50]$, $[0, 220]$ ed $[0, 10,000]$.



Il metodo BDF 2 ed il metodo BDF 3

I metodi Backward Differentiation Formulae (BDF) sono metodi multi-step impliciti, basati su tecniche alle differenze finite per approssimare numericamente le derivate. Il metodo BDF 2 è un metodo multistep ad $s = 2$ passi, L-stabile e di ordine $p = 2$, il metodo BDF 3 è un metodo multistep ad $s = 3$ passi, di ordine $p = 3$ e θ -stabile con angolo $\theta \approx 86.03^\circ$.

Sebbene il metodo BDF 3 sia più preciso del metodo BDF 2, la mancanza della A-stabilità è un prezzo di solito alto da pagare su un problema così stiff, tuttavia in questo caso non sorgono problemi con BDF 3. Ciò è dovuto al fatto che siccome gli autovalori della matrice M sono tutti reali (e non strettamente complessi) saranno sempre inclusi nel cono di stabilità dei metodi BDF, il quale ha l'asse reale come asse di simmetria. Tuttavia, esistono casi specifici dove BDF 2 restituisce errori ottimi grazie alla sua L-stabilità mentre BDF 3 restituisce soluzioni completamente prive di

significato. Il metodo BDF 1 coincide con il metodo di Eulero Implicito, di cui discuteremo in seguito in un paragrafo a sé stante.

I metodi BDF richiedono di risolvere ad ogni iterazione un sistema lineare di dimensione 15×15 con matrice costante, ed è pertanto possibile ottimizzare mediante fattorizzazione LU .

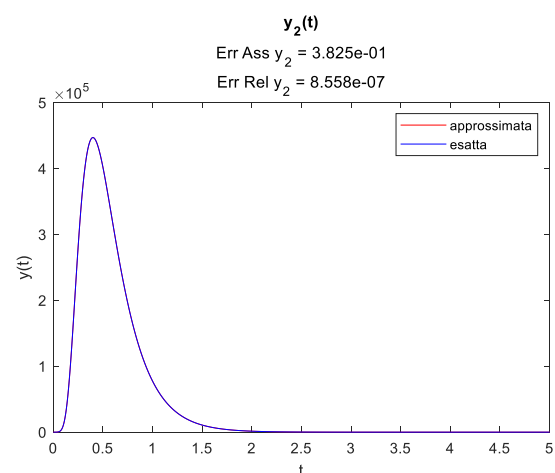
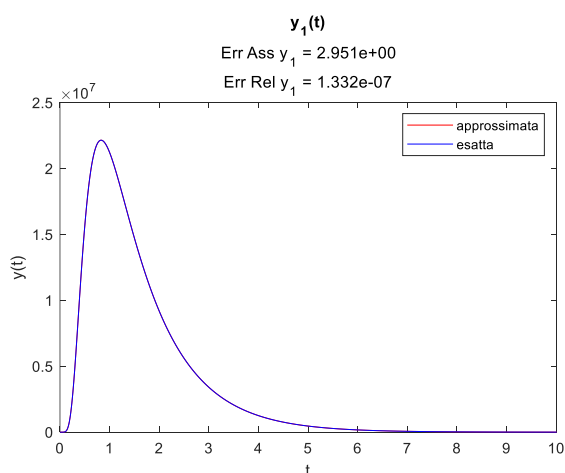
Sebbene i metodi BDF siano i metodi multistep per eccellenza da applicare su problemi caratterizzati da una forte instabilità numerica, in situazioni come questa dove il coefficiente di Stiffness è oltre i 200 è decisamente più conveniente optare per i metodi di Runge Kutta. In effetti, la famiglia dei metodi di Runge Kutta non deve sottostare al teorema della seconda barriera di Dahlquist, il quale afferma che non può esistere un metodo multistep (lineare) A-stabile di ordine superiore a 2.

Segue pertanto che, se per la risoluzione numerica vogliamo utilizzare un metodo multistep che appartenga alla famiglia degli L-stabili, allora al più potremmo sperare in un metodo L-stabile di ordine 2, che è meno di quanto offra il metodo di Radau-IIA con $s = 3$ stadi oppure $s = 2$ stadi. Per quanto riguarda specificatamente i metodi BDF, le implementazioni BDF considerate in questo progetto utilizzano un passo costante. Esistono formulazioni BDF su mesh non omogenee, ma richiedono coefficienti dipendenti dai rapporti tra i passi precedenti e una gestione più articolata. Segue una tabella contenente gli errori, parametri iniziali: $t_0 = 0$, $T = 100$.

BDF 2, $T = 100$			
Numero nodi	Passo h	Err. Ass	Err. Rel
1,000	0.1	1.33083e+06	6.01779e-02
10,000	0.01	2.80888e+04	1.26812e-03
100,000	0.001	2.87072e+02	1.29603e-05
1,000,000	0.0001	2.87657e+00	1.29867e-07

BDF 3, $T = 100$			
Numero nodi	Passo h	Err. Ass	Err. Rel
1,000	0.1	8.46481e+05	3.82764e-02
10,000	0.01	2.79671e+03	1.26262e-04
100,000	0.001	2.95279e+00	1.33308e-07
1,000,000	0.0001	2.96966e-03	1.34070e-10

Detto ciò, i metodi BDF hanno comunque difficoltà ad approssimare $y_{15}(t)$ se h è troppo grande, il motivo è che se una funzione ha una violenta decrescita esponenziale il suo andamento è meglio catturato da un polinomio interpolante, piuttosto che da una funzione che approssima le sue derivate (le quali sono un fenomeno di linearizzazione). Seguono due grafici delle approssimazioni delle funzioni $y_1(t)$ ed $y_2(t)$, tramite il metodo BDF 3 con parametri $T = 100$, $h = 0.001$.



Il metodo di Crank-Nicolson

Il metodo di Crank-Nicolson può essere derivato in vari modi, uno di questi è mediante integrazione numerica basata sul metodo dei trapezi. È un metodo di Runge-Kutta diagonalmente implicito a $s = 2$ stadi di ordine $p = 2$ ed A-stabile. Noi lo abbiamo implementato in versione con mesh non omogenea. Il metodo di Crank-Nicolson coincide tra le altre cose con un metodo di Gauss-Legendre ad $s = 1$ stadi, ma solo quando questi sono applicati su sistemi *lineari* di ODE.

Il fatto che il metodo sia solo *diagonalmente* implicito consente di ottimizzare la risoluzione dei sistemi lineari che compariranno durante il ciclo sulla mesh, in effetti, sfruttando questo fatto, anziché risolvere un sistema lineare 30×30 è possibile andare a risolvere 2 sistemi lineari di dimensione 15×15 (in realtà, per come è fatto il Tableau di Butcher del metodo di Crank-Nicolson, è sufficiente risolverne uno solo), il vantaggio è evidente, calcolando che in condizioni normali la risoluzione di un sistema lineare $n \times n$ costa di $\theta(n^3)$ flops, risolvere un sistema 30×30 richiede circa $30^3 = 27,000$ flops, mentre risolvere due sistemi 15×15 costa circa $2 \cdot 15^3 = 6,750$ flops.

La mancanza della L-stabilità può essere osservata (come per il caso di Gauss-Legendre) sulla 15-esima componente: se si sceglie un passo di discretizzazione h troppo grande, la soluzione numerica introdurrà delle oscillazioni, che comunque poi verranno smorzate grazie alla A-stabilità.

Nel caso di Crank-Nicolson, ciò lo si può esibire anche analiticamente. Mettiamoci pertanto in dimensione 1 ed analizziamo il comportamento di $y_{15}(t)$. La regione di stabilità del metodo di Crank-Nicolson si ricava essere:

$$R(z) = \frac{1 + \frac{z}{2}}{1 - \frac{z}{2}}, \quad \text{con } z = h\lambda \in \mathbb{C}, \quad u_n = R(z) \cdot u_{n-1}.$$

Segue che il metodo sia in effetti A-stabile dal momento che $\forall \lambda \in \mathbb{C}$ con $\text{Re}(\lambda) \leq 0$ accade che $|R(h\lambda)| \leq 1$, e di conseguenza eventuali errori introdotti vengono smorzati e non amplificati, dal momento che:

$$u_n = (R(h\lambda))^n \cdot y_0.$$

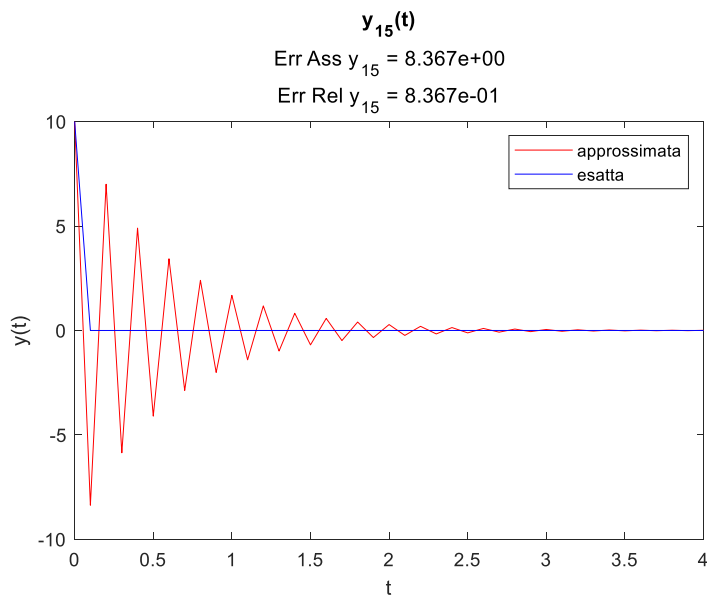
Tuttavia, il metodo non è L-stabile poiché:

$$\lim_{z \rightarrow -\infty} R(z) = \lim_{z \rightarrow -\infty} \frac{1 + \frac{z}{2}}{1 - \frac{z}{2}} = -1 \neq 0$$

Pertanto, se $z = h\lambda$ è negativo e grande in modulo (cioè se il passo h è troppo grande e l'autovalore λ è negativo e molto grande in modulo, tipico appunto dei transitori veloci) il metodo introduce oscillazioni. In effetti, lavorando con $y_{15}(t)$, se $\lambda = -225$, $h = 0.01$, otteniamo:

$$R(h\lambda) = R(-225 \cdot 0.01) = R(-2.25) = \frac{1 + \frac{-2.25}{2}}{1 - \frac{-2.25}{2}} \approx -0.0588 \Rightarrow$$
$$u_n = (R(h\lambda))^n \cdot y_0 = (-0.0588)^n \cdot y_0 = (-1)^n \cdot (0.0588)^n \cdot y_0$$

Di conseguenza, se h è troppo grande, il metodo introduce delle oscillazioni, che però vengono subito smorzate, come si può osservare anche nel seguente grafico ottenuto per $h = 0.1$:



$$t_0 = 0, \quad T = 1,000$$

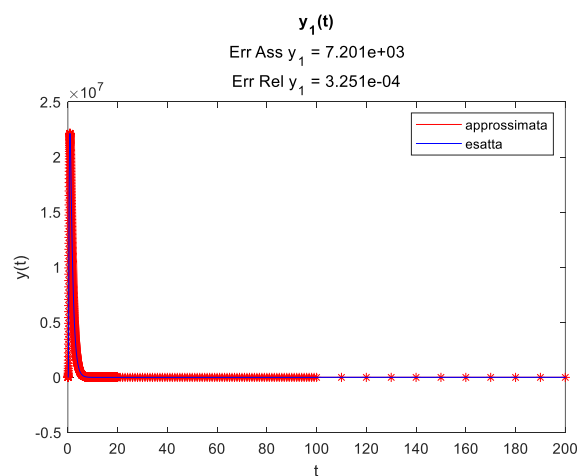
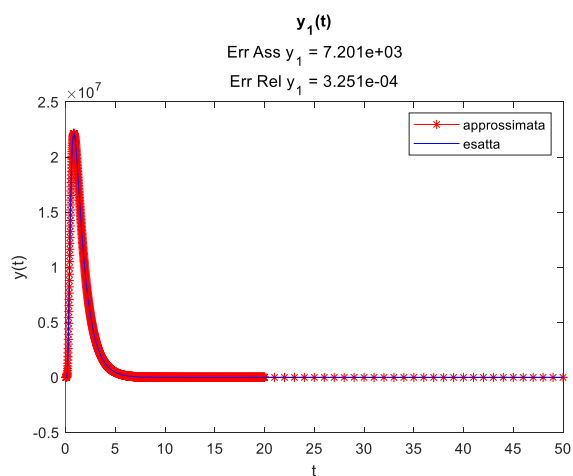
$$h_1 = 0.1, \quad h_2 = 0.1, \quad h_3 = 10$$

La mancanza della L-stabilità introduce delle oscillazioni negli istanti iniziali che però vengono subito smorzate grazie alla A-stabilità.

Diamo di seguito una tabella degli errori, ricordando che la mesh implementata è di tipo non omogeneo. Di seguito, $t_0 = 0$, $T = 10,000$.

Num. nodi	Err. Ass	Err. Rel.	Scelta passi: h_1 in $[t_0, 20]$, h_2 in $[20, 100]$, h_3 in $[100, T]$
290	7.44008e+05	3.36428e-02	$h_1 = 0.1$, $h_2 = 1$, $h_3 = 1000$
2,900	7.20198e+03	3.25146e-04	$h_1 = 0.01$, $h_2 = 0.1$, $h_3 = 100$
20,180	7.19316e+01	3.24747e-06	$h_1 = 0.001$, $h_2 = 1$, $h_3 = 100$
21,791	7.19316e+01	3.24747e-06	$h_1 = 0.001$, $h_2 = 0.1$, $h_3 = 10$
100,000	7.44008e+05	3.36428e-02	$h = 0.1$, mesh omogenea
1,000,000	7.20198e+03	3.25146e-04	$h = 0.01$, mesh omogenea

Rappresentiamo inoltre il grafico di $y_1(t)$, con parametri: $t_0 = 0$, $T = 10,000$, $h_1 = 0.01$, $h_2 = 1$, $h_3 = 10$.



Il metodo di Eulero Implicito

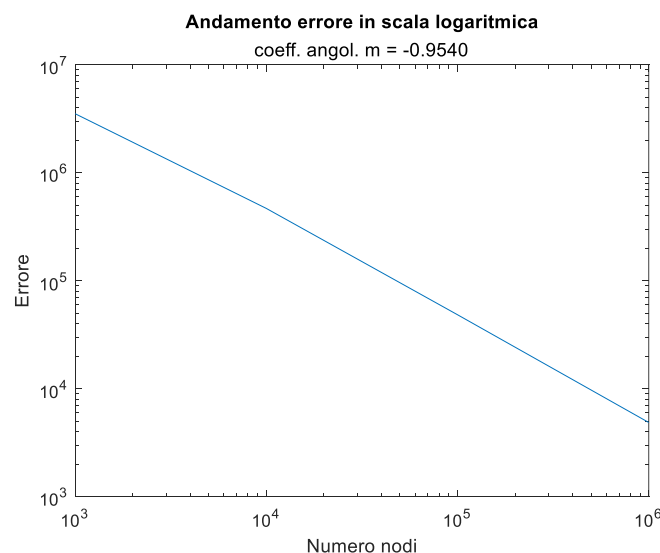
Il metodo di Eulero Implicito è un metodo L-stabile di ordine $p = 1$. Esso coincide con un metodo di Radau-IIA ad uno stadio, ed anche come un metodo BDF 1 ad un passo, può inoltre essere considerato un metodo diagonalmente implicito. Il metodo di Eulero implicito è pertanto un metodo molto stabile, tuttavia, è anche un metodo a convergenza molto lenta: aumentando di un fattore 10 il numero di nodi, l'errore decade di solo un fattore 10^1 . Di contro, però, è un metodo dal costo computazionale molto basso, ad ogni passo ci richiede di risolvere un solo sistema lineare di equazioni di dimensione 15×15 , e come sopra, è possibile anche utilizzare la fattorizzazione LU . Noi lo abbiamo implementato solo in mesh omogenea. Data la sua ridotta velocità di convergenza, di solito non è un metodo che si tende ad usare. Segue una tabella contenente gli errori.

$t_0=0, \quad T = 100$			
Numero nodi	Passo h	Err. Ass	Err. Rel
1000	0.1	3.53628e+06	1.59905e-01
10,000	0.01	4.68461e+05	2.11495e-02
100,000	0.001	4.84397e+04	2.18689e-03
1,000,000	0.0001	4.86061e+03	2.19440e-04

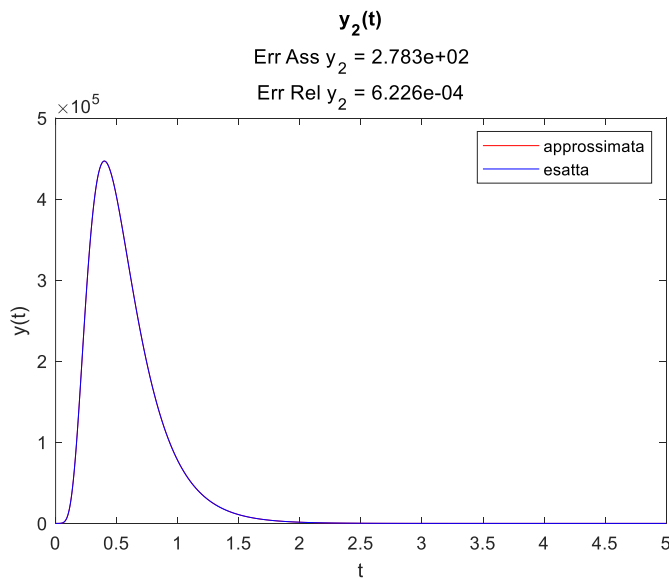
Dal momento che il metodo di Eulero Implicito è un metodo dal costo computazionale molto basso e che pertanto introduce poco rumore macchina nella risoluzione dei sistemi lineari, la progressione degli errori risulta essere molto precisa, è possibile qui osservare bene come:

$$e = \max_{n=0,\dots,N} \|y(t_n) - u(t_n)\|_2 = O(h^1)$$

e che pertanto, diminuendo h di un fattore 10, l'errore diminuisca di un fattore 10^1 . Analizziamo pertanto l'andamento dell'errore in scala logaritmica:



Qui è possibile notare che mettendo in scala logaritmica il numero di nodi (sulle ascisse) e gli errori da noi trovati (sulle ordinate) quello che otteniamo è con buona approssimazione una retta dal coefficiente angolare di $m = -0.9540$, il che è certamente un buon risultato, dal momento che per il metodo di Eulero Implicito, essendo un metodo di ordine $p = 1$, il grafico teorico atteso dalla relazione tra numero di nodi ed errore assoluto in scala logaritmica dovrebbe essere una retta dal coefficiente angolare teorico di $m_t = -1$.



In ogni caso, sebbene il metodo di Eulero Implicito abbia solo ordine 1, anche con esso si può approssimare bene la soluzione.

A sinistra il grafico di $y_2(t)$, approssimata fissando i parametri $t_0 = 0$, $T = 100$, $h = 0.0001$

Il metodo di Pareschi Russo

Il metodo di Pareschi-Russo è un metodo di Runge Kutta a $s = 2$ stadi, diagonalmente implicito, di ordine $p = 2$. Fissato un parametro $R \in \mathbb{R}$, il suo Tableau di Butcher è il seguente:

R	R	0
$1 - R$	$1 - 2R$	R
	$\frac{1}{2}$	$\frac{1}{2}$

$\underline{c}$	A
	$\underline{b^T}$

Inoltre, il metodo si può dimostrare essere:

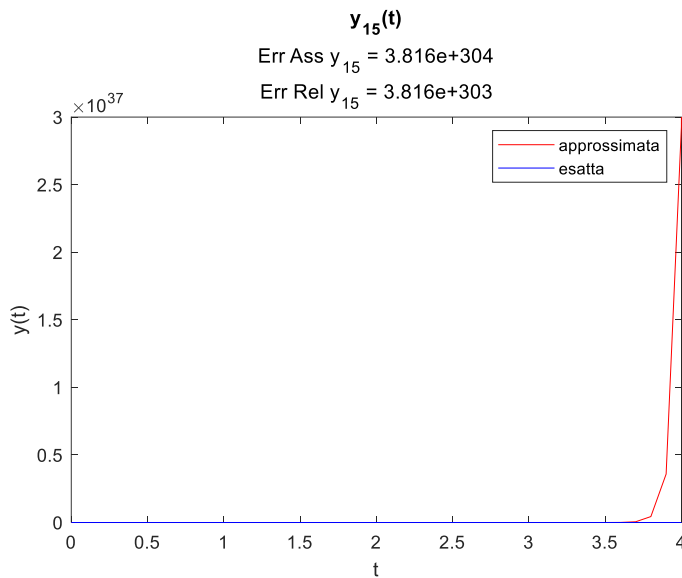
$$A - stabile \Leftrightarrow R \geq \frac{1}{4}$$

$$L - stabile \Leftrightarrow R = 1 \pm \frac{\sqrt{2}}{2}$$

Mentre il metodo risulta essere solo zero stabile qualora $R < \frac{1}{4}$, infine, l'ordine del metodo si dimostra essere $p = 2$ per ogni valore reale del parametro R .

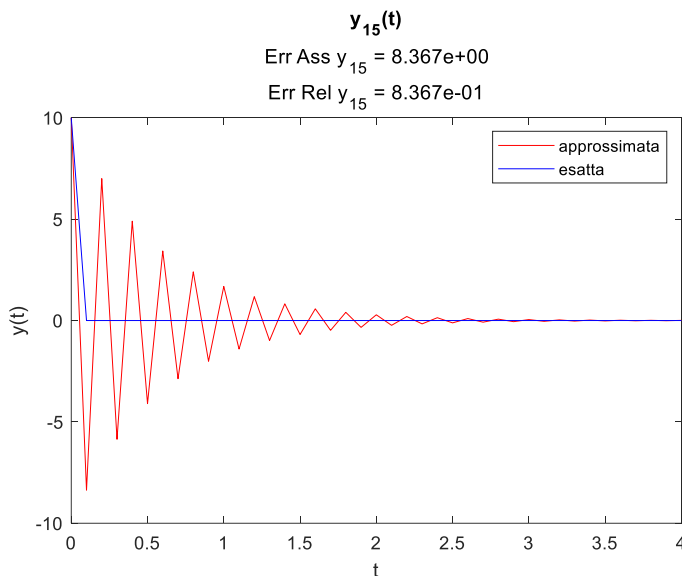
Ancora una volta, la L-stabilità risulta essere una proprietà particolarmente desiderabile per la risoluzione di questo sistema di ODE. Analizziamo la situazione nel dettaglio, assumendo i parametri $t_0 = 0$, $T = 100$, $h = 0.1$, studieremo ancora una volta solo il comportamento di $u_{15}(t)$, dal momento che essa individua la dinamica con l'autovalore maggiore in modulo ed è pertanto quella in cui i fenomeni di instabilità sono più accentuati.

Nel seguito forniremo 3 grafici, tutti e tre prodotti con lo stesso passo $h = 0.1$, cambierà solo il valore del parametro R in modo tale che il metodo sia nella prima immagine solo zero-stabile, poi A-stabile nella seconda immagine, infine L-stabile nella terza.



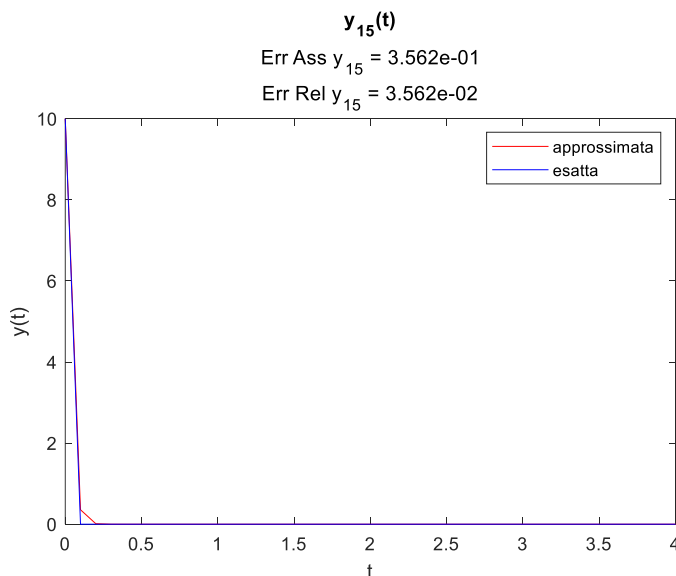
$$T = 100, \quad h = 0.1, \quad R = \frac{1}{8}.$$

Dal momento che $R < 1/4$, il metodo non gode della proprietà di A-stabilità, ma solo di quella di zero-stabilità. La sola zero-stabilità ed il fatto che h sia così grande, da soli non sono sufficienti a garantire la convergenza del metodo, l'errore assoluto esplode fino a 10^{304} la soluzione numerica ottenuta è priva di qualsivoglia significato matematico.



$$T = 100, \quad h = 0.1, \quad R = \frac{1}{4}.$$

Dal momento che $R \geq 1/4$, il metodo gode della A-stabilità. Il fatto che h sia così grande, unito al fatto che il metodo non sia L-stabile, insieme introducono dei fenomeni oscillatori nella soluzione approssimata, destinati comunque a smorzarsi grazie alla A-stabilità. L'errore assoluto commesso è nell'ordine di 10^1 .



$$T = 100, \quad h = 0.1, \quad R = 1 + \frac{\sqrt{2}}{2}$$

Dal momento che $R = 1 \pm \sqrt{2}/2$, il metodo gode della L-stabilità.

Il metodo non introduce fenomeni oscillatori.

In un paio di nodi della mesh, la soluzione approssimata non si sovrappone perfettamente a quella esatta per via del fatto che il passo h è comunque molto grande. L'errore assoluto commesso è nell'ordine di 10^{-1} .

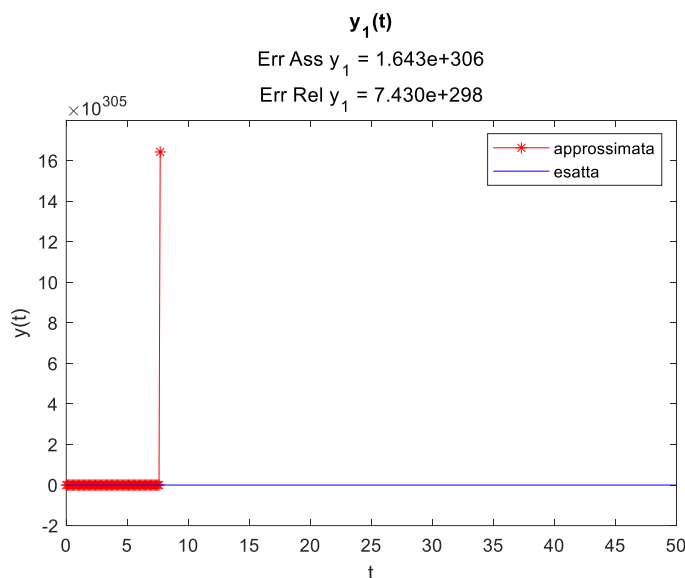
Segue inoltre una tabella degli errori prodotta in situazione di L-stabilità.

$t_0 = 0, \quad T = 100, \quad R = 1 + \sqrt{2}/2$			
Numero nodi	Passo h	Err. Ass	Err. Rel
1,000	0.1	2.00194e+06	9.05245e-02
10,000	0.01	1.01990e+05	4.60452e-03
100,000	0.001	1.17384e+03	5.29951e-05
1,000,000	0.0001	1.18468e+01	5.34843e-07

Metodi RK espliciti ed espliciti adattivi

Questi metodi sono assolutamente sconsigliati per risolvere il problema e di fatto non vanno bene. La regione di stabilità di un metodo esplicito è finita, di conseguenza si rischia che, a meno di prendere un passo h molto molto piccolo, il metodo non converga alla soluzione desiderata. I metodi espliciti non godono della proprietà di A-stabilità, come si può dimostrare in generale. Abbiamo a tal proposito implementato sia il metodo di Runge Kutta 4 (uno tra i più famosi metodi di Runge Kutta espliciti ad $s = 4$ stadi, sebbene non abbia un nome proprio) sia il metodo esplicito adattivo embedded Runge-Kutta-Fehlberg 4(5) (un metodo numerico esplicito ed adattivo molto diffuso).

Nella figura seguente si osserva il grafico di $u_1(t)$ approssimato mediante il metodo RK4 e prodotto fissando i parametri $t_0 = 0, \quad T = 1000, \quad h = 0.1$.

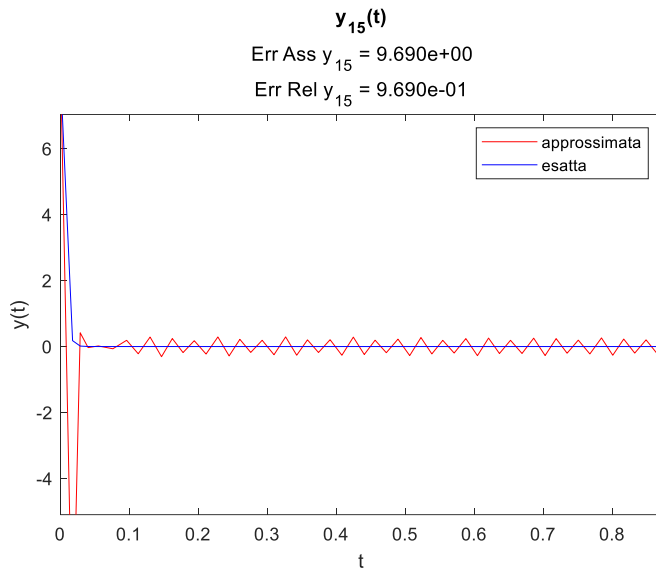


Soluzione numerica prodotta con un metodo esplicito a 4 stadi ed $h = 0.1$.

Essendo esplicito, il metodo non gode della A-stabilità.

La sola zero-stabilità ed il fatto che h sia così grande, da soli non sono sufficienti a garantire la convergenza del metodo, l'errore assoluto esplode fino a 10^{304} e la soluzione numerica ottenuta è priva di qualsivoglia significato matematico.

Abbiamo implementato anche il metodo esplicito adattivo RKF 4(5), il problema rimane comunque la mancanza di A-stabilità, se la tolleranza è alta, allora il metodo, passo dopo passo, crederà di aver stimato l'errore in modo accurato, tuttavia, si tratta di un falso positivo. Il metodo adattivo utilizza infatti due metodi espliciti e tratta quello di ordine superiore come se fosse la soluzione esatta, e, sulla base dello scarto tra le due soluzioni prodotte, decide la lunghezza del passo successivo; il problema è che entrambe le soluzioni numeriche saranno estremamente imprecise, e pertanto il metodo non convergerà alla soluzione esatta, come si può notare nel seguente grafico proposto.

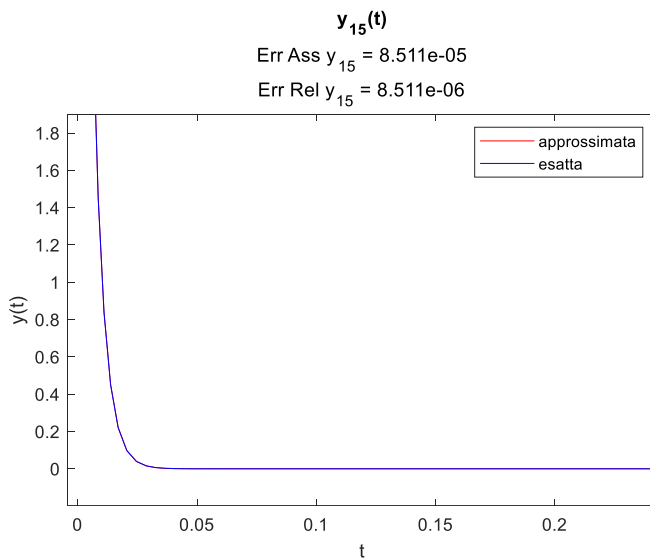


$$t_0 = 0, T = 100, \text{toll} = 10^2$$

Soluzione u_{15} prodotta con un metodo Runge-Kutta-Fehlberg 4(5).

Essendo il metodo non A-stabile poiché esplicito, la soluzione prodotta è affetta da una incertezza molto grande.

Tuttavia, abbassando la tolleranza, il metodo converge comunque con ottima precisione.



$$t_0 = 0, T = 100, \text{toll} = 10^{-3}$$

Soluzione u_{15} prodotta con un metodo Runge-Kutta-Fehlberg 4(5).

Il metodo non è A-stabile, tuttavia, avendo fissato una tolleranza molto bassa, l'errore commesso è comunque ridotto, sebbene comunque rimanga estremamente sconsigliabile utilizzare metodi espliciti per risolvere problemi stiff.

Segue una tabella contenente i vari errori e numero di nodi al variare della tolleranza, fissati i parametri $t_0 = 0, T = 10,000$.

Tolleranza	Numero nodi	Err. Ass.	Err. Rel.	h min	h max
1e+8	611,789	3.13855e+07	1.41695e+00	0.001	0.14525
1e+7	611,791	4.60446e+06	2.07880e-01	0.001	0.10013
1e+6	611,794	2.74500e+05	1.23940e-02	0.001	0.06889
1e+5	611,795	3.72762e+04	1.68297e-03	0.001	0.04726
1e+4	611,796	3.64956e+03	1.64765e-04	0.001	0.03228
1e+3	611,797	3.30129e+02	1.49048e-05	0.001	0.02192
1e+2	611,797	4.15175e+01	1.87442e-06	0.001	0.02109
1e+1	611,797	9.85077e+00	4.44740e-07	0.001	0.01924
1e+0	611,797	2.45056e+00	1.10640e-07	0.001	0.02658
1e-1	611,784	2.97322e-01	1.34231e-08	0.001	0.04553
1e-2	611,789	3.01900e-02	1.36301e-09	0.00001	0.08776
1e-3	611,808	3.00973e-03	1.35879e-10	0.001	0.06063

Si può osservare come sebbene il metodo di Runge-Kutta-Fehlberg sia un metodo ad addirittura $s = 6$ stadi, per ogni fissata tolleranza proposta, il numero di nodi ottenuto è sempre maggiore del metodo di Gauss-Legendre adattivo, il quale lo ricordiamo ha solamente $s = 3$ stadi. Segno che la mancanza di stabilità è pesata particolarmente.

In effetti, osserviamo che per $T = 10,000$, anche fissando una tolleranza altissima di $1e+5$ il metodo ha comunque bisogno di circa 611,000 nodi per abbattere l'errore, si tratta di un numero enorme e che diversi metodi impliciti A-stabili riescono a superare anche su mesh omogenea.

È davvero considerevole osservare come per ottenere un errore dell'ordine di 10^1 , il metodo adattivo di Gauss-Legendre necessiti di solo 70 nodi, mentre il metodo di RKF ne richieda addirittura 611,000.

Infine, si segnala che l'esecuzione del metodo adattivo RKF con $T = 10,000$, anche con una tolleranza di $1e+5$, può richiedere fino a 2 o 3 minuti di calcolo, testimoniando il fatto che il metodo adattivo ha bisogno di ricalcolare al ribasso la lunghezza del passo diverse volte, prima di riuscire a trovare un valore di h che soddisfi il vincolo sulla tolleranza. Ne consegue che, sebbene in generale ci si aspetta che un metodo esplicito abbia un costo computazionale molto economico, dettato dall'assenza di necessità di risolvere sistemi lineari ad ogni passo, tale vantaggio viene qui completamente annullato dai vincoli adattivi sulla tolleranza.

Ricordando infine che la tolleranza è, per lo meno in linea teorica, uno stimatore (per eccesso) dell'errore assoluto, è possibile osservare nella tabella sopra come questo stimatore sia affidabile: in ogni riga l'errore assoluto è inferiore alla tolleranza o comunque avente lo stesso ordine di grandezza. Tuttavia, è anche singolare osservare come l'errore assoluto venga abbattuto senza sostanzialmente che cambi il numero di nodi. Ciò è dovuto al fatto che il metodo non riesce a calcolare un passo h più lungo nella coda dell'intervallo, finendo per creare sostanzialmente una mesh che è con buona approssimazione una mesh omogenea. Il motivo per cui si abbatte l'errore assoluto senza che venga abbattuto il numero di nodi è che l'*argmax* dell'errore assoluto giace comunque negli istanti iniziali della mesh.

È infine interessante osservare i fenomeni oscillatori introdotti quando la tolleranza è troppo alta, segue il grafico di $u_1(t)$, approssimato mediante il metodo RKF 4(5) e prodotta fissando i parametri $t_0 = 0$, $T = 10,000$, $toll = 10^8$.

